Information Retrieval

WS 2022 / **2023**

Lecture 11, Tuesday January 24th, 2023

(Linear Classifiers, Perceptrons, Logistic Regression)

Prof. Dr. Hannah Bast
Chair of Algorithms and Data Structures
Department of Computer Science
University of Freiburg

UNI FREIBURG

Overview of this lecture

- Organizational
  - Experiences with ES 10 — Naïve Bayes
  - Official course evaluation — **Deadline: 29.01.2023**
- Contents
  - Linear classifiers — definition and the math behind
  - Naïve Bayes (again) — is a linear classifier
  - Perceptron — is a very simple linear classifier
  - Logistic Regression — a related, but mathematically well-founded linear classifier
  - **ES11:** solve the exact same task from ES10, but now with logistic regression … and compare the results against NB

Experiences with ES10 1/4

- Summary / excerpts
  - Very little code, but the linear algebra is tricky

    "Really nice to see how a lot can be done in a few lines"

    "I particularly liked doing it based on matrices"

    "It was a nice and challenging exercise, I really liked it"

    "I learned a lot by solving this exercise!"

    "Shouldn't 'man' or 'woman' be added to the stopword list?"

    "The idea behind the algorithm was super simple, but it took a lot of time to find the numpy functions"

    "Matrices + numpy is a hill too tall for me to climb"

    "Very interesting lecture! Prof Bast seems to be very happy being allowed to do so much math … was very contagious"

Experiences with ES10 2/4

- Videos by MrBeast
  - Most of you know him and like his videos, few critical voices

    "Squid game: interesting way to try and spend all the money"

    "His videos are enjoyable, especially the Squid game"

    "I watched MrBeast videos, they are funny and expensive"

    "He puts a lot of effort into learning what people like"

    "Perfectly made to the reach as many people as possible"

    "I like seeing people using money to actively improve the world"

    "Interesting person … but videos are not for me"

    "Not the style and type of content I enjoy"

    My opinion: the fast cuts drive me crazy, I can barely watch it, looks like an expression of severe attention deficit disorder

Experiences with ES10 3/4

- Results **Co**medy, **Ro**mance, **Ho**rror, **Do**cumentary, **Sc**i-Fi
  - Precision: Co 77%, Ro 56%, Ho 71%, Do 72%, Sc 55%
  - Recall: Co 74%, Ro 54%, Ho 75%, Do 81%, Sc 64%
  - Frequency: Co 50%, Ro 20%, Ho 15%, Do 11%, Sc 4%
  - Understand: higher recall than precision for a class means:

    Tendency to predict that class too much
  - Some classes have more specific words than others:

    Horror: killer, mysterious, old, death, night, town, …

    Romance: man, woman, story, family, father, wife, …
  - The word "young" is frequent in all classes except Do **bias alert**

    https://www.youtube.com/watch?v=dIzPhEecXN0&t=602s

Experiences with ES10 4/4

- Which precision is considered "good" ?
  - **Baseline 1:** guess label uniformly at random

    Overall precision: 20% (1 / #classes)
  - **Baseline 2:** always predict the most frequent label

    Overall precision: 50% (frequency of "Comedy")

    Deceptively good results when one class is very frequent
  - **Baseline 3:** pick label c with probability $p_c = |T_c| / |T|$

    This is not better than Baseline 2, and generally worse

    Think about why … this might be an exam question

Official course evaluation

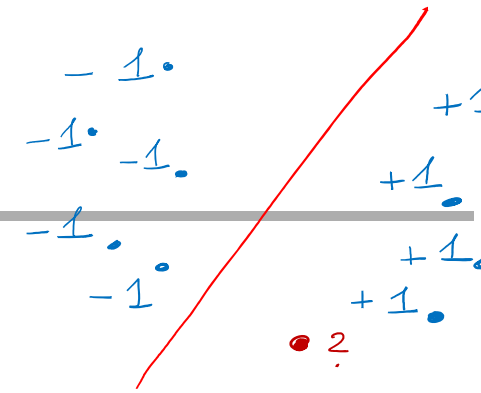
- Instructions
  - You should have received an email from EvaSys Admin on Monday, **January 16** with a link to an evaluation form
  - We are **very** interested in your feedback
  - Please take your time for this and be **honest**, **concrete**, **fair**
  - The free text comments are particularly interesting for us

Please complete until Sunday, January 29 the latest

The evaluation is centralized, and once it is closed, there is nothing we can do about that anymore

If you did not receive any email or if you have any other question, please let us know in the subforum "Evaluation"

Linear Classifiers 1/6



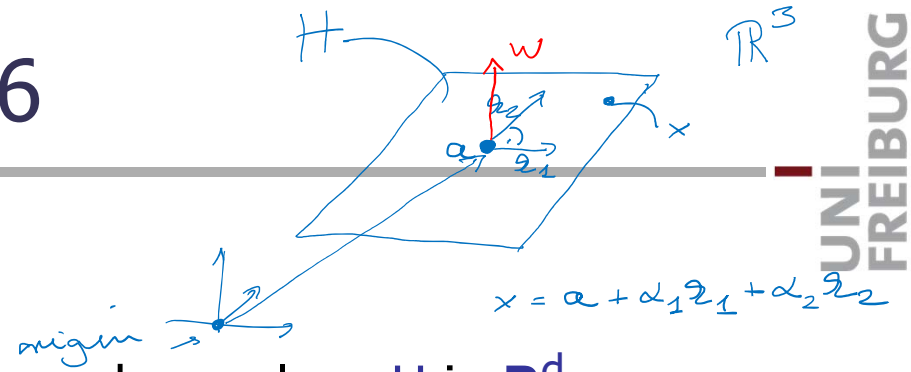
- Framework
  - Objects are vectors in d dimensions
  - Exactly **two** classes, often called + and – or +1 and –1

    See slide 12 for how to generalize to more classes
  - A linear classifier tries to separate the data points by a (d-1)-dimensional hyperplane, as defined on next slide

    For d=2 this means: try to separate by a straight line

    Note that the points may not be fully separable, see slide 13 for what can be done when they are not
  - Predictions are made based on which side of the hyperplane (for d=2 a straight line) the object lies on

Linear Classifiers 2/6



- Hyperplane, definitions
  - Two common ways to define a hyperplane H in $\mathbf{R}^d$

    The two definitions are equivalent … proof on next slide
  - **Definition 1** (by anchor point and basis):

    There is an anchor point $a \in \mathbf{R}^d$ and pairwise orthogonal $h_1, \ldots, h_{d-1} \in \mathbf{R}^d$ such that H consists of all linear combinations $a + \Sigma_i \alpha_i h_i$ for arbitrary $\alpha_1, \ldots, \alpha_{d-1} \in \mathbf{R}$
  - **Definition 2** (by normal vector and offset):

    There is a normal vector $w \in \mathbf{R}^d$ and an offset $b \in \mathbf{R}$ such that H consists of all points $x \in \mathbf{R}^d$ with $w \bullet x = b$

$x, y \in \mathbb{R}^d$, $\alpha \in \mathbb{R}$ $\quad \alpha \cdot (x \bullet y) = (\alpha x) \bullet y = x \bullet (\alpha y)$ $\qquad x \bullet y = \sum_{i=1}^{d} x_i \cdot y_i$

Linear Classifiers 3/6

assume that a is linearly independent from $z_1, \dots, z_{d-1}$ (otherwise H contains the origin)

- Hyperplane, equivalence of these definitions
  - For the proof we use that for any pairwise orthogonal $x_1, \dots, x_k \in \mathbf{R}^d$, we can find $x_{k+1}, \dots, x_d \in \mathbf{R}^d$ such that all $x_1, \dots, x_d$ are pairwise orthogonal

$H := \{x : x = a + \sum_{i=1}^{d-1} \alpha_i \cdot z_i, \ \alpha_1, \dots, \alpha_{d-1} \in \mathbb{R}\}$

Pick w such that $w \perp z_1, \dots, z_{d-1}$

Pick $b := w \bullet a$

$\boxed{H' := \{x : w \bullet x = b\}}$

We want to prove: $H = H'$

$H \subseteq H'$: $x \in H \Rightarrow \exists \alpha_1, \dots, \alpha_{d-1} \in \mathbb{R} : x = a + \sum_{i=1}^{d-1} \alpha_i z_i$

$\Rightarrow w \bullet x = \underbrace{w \bullet a}_{=b} + \sum_{i=1}^{d-1} \alpha_i \underbrace{w \bullet z_i}_{=0 \text{ because } w \perp z_i} = b \Rightarrow x \in H'$

$H' \subseteq H$: $x \in H' \Rightarrow w \bullet x = b$

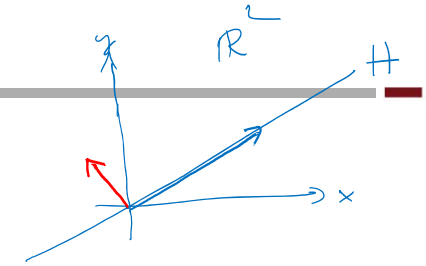
we can write <u>any</u> $x \in \mathbb{R}^d$ as $x = \sum_{i=1}^{d-1} \alpha_i z_i + \underline{\alpha_d \cdot a}$

$\Rightarrow b = \sum_{i=1}^{d-1} \alpha_i \underbrace{w \bullet z_i}_{=0} + \alpha_d \cdot \underbrace{w \bullet a}_{=b} = \alpha_d \cdot b$

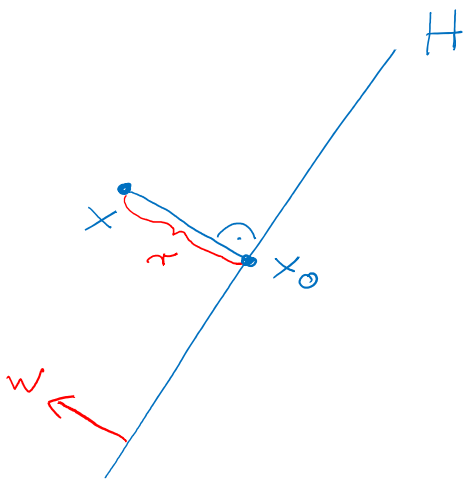
$\Rightarrow \underbrace{\alpha_d = 1}_{\text{then we are done } (x \in H)}$ or $\underbrace{b = 0}$ $\rightarrow a \perp w \Rightarrow a \in H \Rightarrow H$ contains the origin ...

$x \in \mathbb{R}^d \Rightarrow x \bullet x = \sum_{i=1}^{d} x_i^2 = |x|^2$

Linear Classifiers 4/6



- Distance from a point to a hyperplane
  - Let $H = \{ x \in \mathbf{R}^d : w \bullet x = b \}$ be a hyperplane in $\mathbf{R}^d$
  - Then the distance of a point $x \in \mathbf{R}^d$ to H is $|w \bullet x - b| / |w|$
  - The sign of $w \bullet x - b$ says on which side of H lies x



$w_0 = w/|w|$ unit vector pointing in the same direction as w

$\Rightarrow x = x_0 + r \cdot w_0$

$\Rightarrow w \bullet x = \underbrace{w \bullet x_0}_{=b} + r \cdot \underbrace{w \bullet w_0}_{= w \bullet w / |w| = |w|}$

$= b + r \cdot |w|$

$\Rightarrow r = \underbrace{(w \bullet x - b)}_{> 0} / |w| = |w \bullet x - b| / |w|$ (x lies on the side of H where w points)

The case, where x lies on the other side, can be proved analogously

Linear Classifiers 5/6

pick the class with highest prediction value

- Generalization to more classes
  - **Option 1:** Build k classifiers, one for each class, with the i-th one doing the classification: Class i OR not Class i

    Drawback: comparing scores from two different classifiers might not be fair (maybe scores tend to be higher in one)
  - **Option 2:** Build $k \cdot (k - 1) / 2$ classifiers, one for each subset of two classes

    Drawback: For large k, that's a lot of classifiers !
  - **Option 3:** Extend theory of the respective approach to deal with more than two classes directly

    Drawback: Sometimes hard (not for Naïve Bayes though)

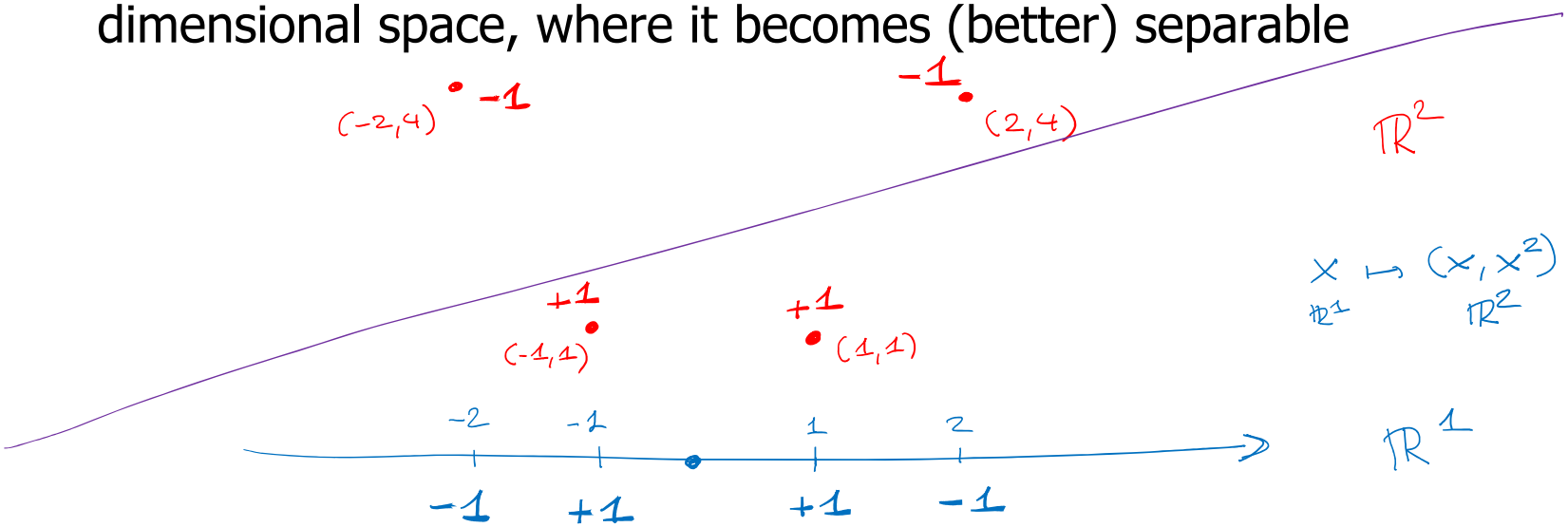
    For ES11, predict the class with the highest score (Option 1)

Linear Classifiers 6/6

- When the data is not linearly separable not relevant for ES11
  - **Option 1:** extend the method to accept "outliers"

    Naïve Bayes does this by definition: it always computes a hyperplane, whether the data is linearly separable or not

  - **Option 2:** suitably transform the data to some higher-dimensional space, where it becomes (better) separable



Naïve Bayes 1/5

- Recap from last lecture
  - Let the vocabulary of all words be $V = \{v_1, \ldots, v_{|V|}\}$

    Beware: in todays lecture, **w** is reserved for normal vectors of hyperplanes, which is why we now denote a word by **v**
  - Recall how NB predicts the probability of a class C for d

    $Pr(C=c \mid D=d) = \Pi_{v_i \in D}\, p_{ic} \cdot p_c \,/\, Pr(D=d)$

    where $p_{ic} = Pr(W=v_i \mid C=c)$ and the factor is taken multiple times for multiple occurrence of v_i in document d
  - We can equivalently write this as

    $Pr(C=c \mid D=d) = \Pi_{i=1,\ldots,|V|}\, p_{ic}^{tf_i} \cdot p_c \,/\, Pr(D=d)$

    where tf_i is the number of occurrences of $v_i \in D$

Naïve Bayes 2/5

- Using linear algebra
  - Prediction formula from the previous slide

    $Pr(C=c|D=d) = \Pi_{i=1,\ldots,|V|}\ p_{ic}^{tf_i} \cdot p_c\ /\ Pr(D=d)$

    where tf_i is the number of occurrences of v_i in D
  - Take the logarithm and use shorthand $P(c|d)$ for $Pr(C=c|D=d)$

    $\ln Pr(c|d) = \Sigma_{i=1,\ldots,|V|}\ tf_i \cdot \ln p_{ic} + \ln p_c - \ln Pr(D=d)$
  - Write d and the p_{ic} for class c as $1 \times |V|$ vectors $\mathbf{d}$ and $\mathbf{p_c}$

    $\mathbf{d} = (tf_1, \ldots, tf_{|V|})$ $\quad \mathbf{p_c} = (\ln p_{1c}, \ldots, \ln p_{|V|c})$
  - Then we can write $\ln Pr(c|d)$ via a simple dot product:

    $\ln Pr(c|d) = \mathbf{d} \bullet \mathbf{p_c} + \ln p_c - \ln Pr(D=d)$

Naïve Bayes 3/5

$\ln x - \ln y = \ln \frac{x}{y}$

$b = 0 \iff p_A = p_B$

- Two-class NB is a linear classifier
  - Assume our two classes are called A and B, then we have

    $\ln \Pr(A|d) = \mathbf{d} \bullet \mathbf{p_A} + \ln p_A - \ln \Pr(D=d)$

    $\ln \Pr(B|d) = \mathbf{d} \bullet \mathbf{p_B} + \ln p_B - \ln \Pr(D=d)$
  - Then $\Pr(A|d) > \Pr(B|d)$ if and only if

    $\ln \Pr(A|d) - \ln \Pr(B|d) = \mathbf{d} \bullet (\mathbf{p_A} - \mathbf{p_B}) + \ln (p_A / p_B) > 0$

    where $\mathbf{p_A} - \mathbf{p_B} = (\ln (p_{1A}/p_{1B}), \ldots , \ln (p_{|V|A}/p_{|V|B}))$
  - Define $b = -\ln (p_A / p_B)$ and $\mathbf{w} = \mathbf{p_A} - \mathbf{p_B}$
  - Then NB predicts class A for document $\mathbf{x}$ if and only if

    $\mathbf{w} \bullet \mathbf{x} - b > 0$

H contains origin $\Longleftrightarrow$ $b = 0$

Naïve Bayes 4/5

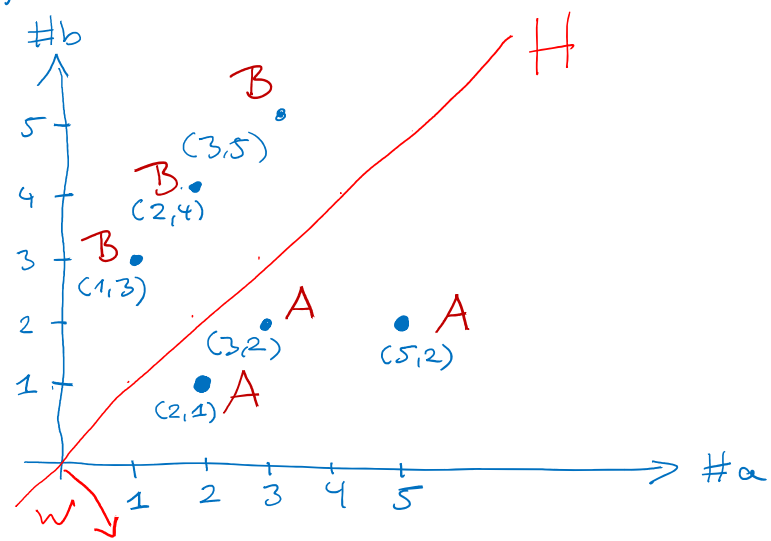
$p_A = \frac{1}{2}$, $p_B = \frac{1}{2}$

$p_{aA} = \frac{2}{3}$, $p_{bA} = \frac{1}{3}$

$p_{aB} = \frac{1}{3}$, $p_{bB} = \frac{2}{3}$

- Our toy example from Lecture 10
  - Now express geometrically, in terms of w and b

$(\#a, \#b)$		
(2,1)	aba	A
(5,2)	baabaaa	A
(3,5)	bbaabbab	B
(3,2)	abbaa	A
(1,3)	abbb	B
(2,4)	bbbaab	B



$$w = \left(\ln \frac{p_{aA}}{p_{aB}}, \ln \frac{p_{bA}}{p_{bB}}\right)$$

$$= \left(\ln 2, \underbrace{\ln \frac{1}{2}}_{= -\ln 2}\right)$$

$$= \ln 2 \cdot (1, -1)$$

$$b = -\ln \frac{p_A}{p_B} = -\ln 1 = 0$$

$$x = (\#a, \#b)$$

$$x \in A \Longleftrightarrow$$

$$w \bullet x = \ln 2 \,(\#a - \#b) > 0$$

$$\Longleftrightarrow \#a > \#b$$

Naïve Bayes 5/5

- Avoiding the extra b term
  - We can avoid the extra term, by adding another dimension
  - Instead of $|V| \times 1$ vectors, consider $(|V| + 1) \times 1$ vectors:

    $\mathbf{d} = (tf_1, \ldots, tf_{|V|}, 1) \quad \mathbf{p_c} = (\ln p_{1c}, \ldots, \ln p_{|V|c}, \ln p_c)$
  - Then the expression for $\ln P(c|d)$ simplifies to:

    $\ln Pr(c|d) = \mathbf{d} \bullet \mathbf{p_c} - \ln Pr(D=d)$
  - For two classes, we then have that $\ln Pr(A|d) > Pr(B|d)$ iff

    $\mathbf{d} \bullet (\mathbf{p_A} - \mathbf{p_B}) > 0$
  - Defining $\mathbf{w} = \mathbf{p_A} - \mathbf{p_B}$ this is equivalent to

    $\mathbf{w} \bullet \mathbf{d} > 0 \ldots \mathbf{w} = (\ln (p_{1A}/p_{1B}), \ldots, \ln (p_{|V|A}/p_{|V|B}), \ln (p_A|p_B))$

Perceptrons 1/4

- Intuition
  - A perceptron is a linear classifier that **iteratively** computes the **w** that defines the hyperplane used for prediction

    As on slide 18, we assume that the **b** is one component of the **w**, by adding one dimension to the input vectors and setting the entry in that dimension to **1** for all input vectors
  - During training, **w** is greedily improved in each iteration
  - If the training set is linearly separable, the algorithm provably terminates, see slide 22
  - The Perceptron is as old as 1958

    Today, its relevance is merely historical and didactic: it's really simple and it's surprising that something so simple works

Perceptrons 2/4

- Algorithm
  - Initialization: set $w = 0$ (all-zero vector, including the "b")
  - Then iterate over the objects from the training set in random order, and for each object x do the following:

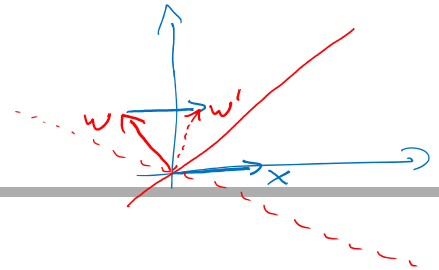
    If $w \bullet x$ gives the right prediction, do nothing

    If $w \bullet x$ gives the wrong prediction, update w as follows:

    - if $w \bullet x < 0$: $w \leftarrow w + x$
    - if $w \bullet x \geq 0$: $w \leftarrow w - x$
  - Repeat until all predictions correct or after fixed #iterations

    Note that it makes sense to iterate over the training set many times (because the w is different after one complete iteration)

Perceptrons 3/4



- Why does w + x or w – x make sense?
  - Intuition: **w** is the direction towards "positive" objects

    So if an object is labeled +, but is wrongly classified as –, it makes sense to correct **w** in the direction of **x** … and vice versa
  - More specifically, let us verify that the value of $w \bullet x$ is closer to what it should be after an update step
  - Case: $w \bullet x < 0$, but we want to have $w \bullet x > 0$

    $w' = w + x \;\rightarrow\; w' \bullet x = w \bullet x + \underbrace{x \bullet x}_{|x|^2 > 0} > w \bullet x$

    The correction goes in the right direction, towards $w \bullet x > 0$
  - Case: $w \bullet x > 0$, but we want to have $w \bullet x < 0$

    $w' = w - x \;\rightarrow\; w' \bullet x = w \bullet x - x \bullet x < w \bullet x$

Perceptrons 4/4

- Convergence
  - On the previous slide we have seen that the correction in each iteration makes sense
  - The hope is that the various updates (for the various **x**) do not cancel each other out, and eventually all predictions are correct on the training set (if the data is linearly separable)
  - **Theorem:** If the training data is linearly separable by some **w**, let $M = \max |x_i|$ and let ϵ be the minimal distance of a training sample from the hyperplane defined by w. Then the perceptron algorithm converges in $O(M^2 / \epsilon^2)$ iterations
  - Instead of proving this, let us look at a related, mathematically much better founded method: Logistic Regression (LR)

LR is described on the next slides and is the topic of ES11

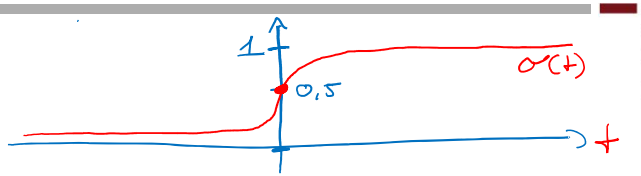
Logistic Regression 1/6

$\left[\frac{1}{x}\right]' = -\frac{1}{x^2}$

- Sigmoid function

$\sigma(t) = \frac{1}{1+e^{-t}}$

$\in (0,1)$



  - Before we can define what logistic regression is and does, we need to know the **sigmoid** function and some of its properties

$\sigma(t) := 1 / (1 + e^{-t})$ for all $t \in \mathbf{R}$

(P1) $\lim_{t\to-\infty} \sigma(t) = 0$, $\lim_{t\to\infty} \sigma(t) = 1$, $\sigma(0) = 0.5$, $\sigma(t) \in [0, 1]$

(P2) $\sigma(-t) = 1 - \sigma(t)$ for all t

(P3) $\sigma'(t) = \sigma(t) \cdot (1 - \sigma(t))$ for all t

Proof of P1: just plug it in

Proof of P2: $\sigma(-t) = \frac{1}{1+e^{t}} = \frac{e^{-t}}{1+e^{-t}} = \frac{1+e^{-t}}{1+e^{-t}} - \frac{1}{1+e^{-t}} = 1 - \sigma(t)$

Proof of P3: $\sigma'(t) = -\frac{1}{(1+e^{-t})^2} \cdot (-e^{-t}) = \sigma(t) \frac{e^{-t}}{1+e^{-t}}$

$= \sigma(t) \cdot (1 - \sigma(t))$ ∎

Logistic Regression 2/6

- Terminology
  - Denote by N the dimension of the input space, including the additional dimension for the bias term b

    For our text data, N is the size $|V|$ of the vocabulary **plus one**
  - Denote by $\mathbf{x_1}, \ldots, \mathbf{x_n} \in \mathbf{R}^N$ the n training examples
  - Denote by $y_1, \ldots, y_n \in [0, 1, \ldots, K-1]$ their labels (y_i label for $\mathbf{x_i}$)

    K is the number of classes; in this lecture we will explain logistic regression for two classes, so $y_1, \ldots, y_n \in \{0, 1\}$

    Logistic regression can be generalized to an arbitrary number of classes, it is then called **multinomial logistic regression**

    For ES11, you can and should simply use one binary LR for each class and then predict the class with the highest score

Logistic Regression 3/6

$p_A = (\ln p_{1A}, \ldots, \ln p_{NA})$
$\Rightarrow \ldots \Rightarrow$ this gave us $\ln P(A|d)$

- Probabilistic model
  - Just like any other linear classifier, logistic regression assumes an underlying hyperplane defined by $\mathbf{w} \in \mathbf{R}^N$
  - The assumption is that for some fixed $\mathbf{w} \in R^N$, for an arbitrary given object $\mathbf{x} \in \mathbf{R}^N$, its label $y \in \{0, 1\}$ is generated as follows:

    $\Pr(y = 1|x) = \sigma(w \bullet x)$ and $\Pr(y = 0|x) = 1 - \sigma(w \bullet x)$
  - In comparison, Naïve Bayes with two classes does the following:

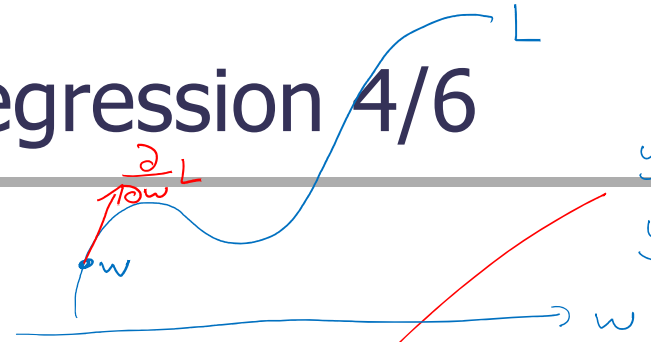
    $\Pr(y=1|x) \sim \exp(w_A \bullet x)$ and $\Pr(y=0|x) \sim \exp(w_B \bullet x)$

    The const. factor is chosen such that $\Pr(y=1|x) + \Pr(y=0|x) = 1$

    In deep-learning speak, this corresponds to normalizing $w_A \bullet x$ and $w_B \bullet x$ using the so-called **Softmax** function

    Both NB and LR push probabilities away from middle to 0 or 1

Logistic Regression 4/6



$y=0: \quad 1-\sigma(w \bullet x)$

$y=1: \quad \sigma(w \bullet x)$

- Update rule
  - Just like for the Perceptron, in Logistic Regression we also start with an arbitrary $\mathbf{w} \in \mathbf{R}^N$ (e.g. zero) and then update it iteratively
  - For the update rule, we use maximum likelihood estimation (MLE), as introduced in Lecture 10
  - Write the probability that label y is correct for x with LR as

    $$\Pr(y|x) = \sigma(w \bullet x)^y \cdot (1 - \sigma(w \bullet x))^{1-y}$$

  - We want to find w to maximize $\Pr(y|x)$, or equivalently $\ln \Pr(y|x)$

    $$L = \ln \Pr(y|x) = y \cdot \ln \sigma(w \bullet x) + (1 - y) \cdot \ln (1 - \sigma(w \bullet x))$$

  - There is no closed-form solution for this, so we compute the partial derivatives with respect to the components of **w**, called <u>gradient</u>, and update **w** in that direction (trying to "climb the hill")

Logistic Regression 5/6

$$[\ln \sigma(x)]' = \frac{1}{\sigma(x)} \cdot \sigma'(x)$$

chain rule

$$[\ln \sigma(w \bullet x)]' = \frac{1}{\sigma(w \bullet x)} \sigma'(w \bullet x) \cdot x$$

- Calculating the gradient $\partial L/\partial \mathbf{w}$
  - **Lemma:** $\partial L/\partial \mathbf{w} = (y - \sigma(\mathbf{w} \bullet \mathbf{x})) \cdot \mathbf{x}$ for $y \in \{0, 1\}$
  - Proof: the proof is relatively simple, using the following

$L = y \cdot \ln \sigma(\mathbf{w} \bullet \mathbf{x}) + (1 - y) \cdot \ln (1 - \sigma(\mathbf{w} \bullet \mathbf{x}))$ slide 26

$\sigma'(t) = \sigma(t) \cdot (1 - \sigma(t))$ for all t slide 23

$\partial/\partial \mathbf{w}\ \mathbf{w} \bullet \mathbf{x} = \mathbf{x}$ "trivial"

$$\frac{\partial}{\partial w} \left(y \cdot \ln \sigma(w \bullet x) + (1-y) \cdot \ln (1 - \sigma(w \bullet x)) \right)$$

chain rule twice

$$= y \cdot \frac{1}{\cancel{\sigma(w \bullet x)}} \cdot \cancel{\sigma(w \bullet x)} \cdot (1 - \sigma(w \bullet x)) \cdot x + (1-y) \cdot \frac{1}{\cancel{1 - \sigma(w \bullet x)}} \cdot \left(-\sigma(w \bullet x) \cdot \cancel{(1 - \sigma(w \bullet x))} \cdot x \right)$$

$$= y \cdot (1 - \sigma(w \bullet x)) \cdot x - (1-y) \cdot \sigma(w \bullet x) \cdot x$$

$$= (y - \sigma(w \bullet x)) \cdot x \quad \text{for } y = \{0, 1\} \qquad \blacksquare$$

Logistic Regression 6/6

- Update rule and batching
  - To update $\mathbf{w}$ using one sample $\mathbf{x} \in \mathbf{R}^N$ with label $y \in \{0, 1\}$, we simply do the following, where α is a fixed tuning parameter called the learning rate:

    $$\mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot (y - \sigma(\mathbf{w} \bullet \mathbf{x})) \cdot \mathbf{x}$$

    Note the similarity to the Perceptron update rule on slide 20: it also updates $\mathbf{w}$ in the direction of $\mathbf{x}$, just with a different factor, namely +1 for y = 1 and −1 for y = 0

  - In practice, it is both more efficient and converges faster, when we update $\mathbf{w}$ using several samples $\mathbf{x_1}, \ldots, \mathbf{x_m}$ at once:

    $$\mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot 1/m \cdot \Sigma_{i=1,\ldots,m} (y_i - \sigma(\mathbf{w} \bullet \mathbf{x_i})) \cdot \mathbf{x_i}$$

    This is called <u>batching</u> and m is called the batch size; for ES11 experiment yourself to find a good batch size

References

- Wikipedia
  - http://en.wikipedia.org/wiki/Linear_classifier
  - http://en.wikipedia.org/wiki/Perceptron
  - https://en.wikipedia.org/wiki/Logistic_regression